NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

UCS 2312 Data Structures Lab

Assignment 5: BSTADT and its application

Date of Assignment: 20.10.2023

Create an ADT for the binary search tree data structure with the following functions. Each node which consists of integer data, address of left and right children.

[CO2, K3]

a. insertBST(t,data) – insert data into BST
b. inorder(t) – display the tree using inorder traversal
c. preorder(t) – display the tree using preorder traversal
d. postorder(t) – display the tree using postorder traversal
e. levelorder(t) – display the tree hierarchically
f. findmin(t)– returns the minimum element in the tree
g. search(t,key) – returns the element found, otherwise returns NULL
h. delete(t,elt) – delete the given elt from tree

1. Demonstrate the BSTADT with the following test case

Insert(t,29)
Insert(t,23)
Insert(t,4)
Insert(t,13)
Insert(t,39)
Insert(t,31)
Insert(t,45)
Insert(t,56)
Insert(t,49)
Inorder(t) → 4,13,23,29,31,39,45,49,56
Levelorder(t)→ 1st Level → 29
2nd level → 23, 39
3rd Level →4, 31, 45
4th Level →13, 56
5th Level → 49
Findmin(t) → 4
Find(t, 13) → Found, value is 3
Find(t,3) → Not found

Best practices to be followed:

- Design before coding
- Usage of algorithm notation
- Use of multi-file C program
- Versioning of code

NAME : S.KARTHIKEYAN 3122 22 5001 056

CSE B

CODE:

Bstadt.h

```c
#include <stdio.h>
#include <stdlib.h>

struct tree
{
    int data;
    struct tree *left,*right;
};


struct tree *insert(struct tree *t,int x)
{
    if (t==NULL)
    {
        t=(struct tree *)malloc(sizeof(struct tree));
        t->data=x;
        t->left=t->right=NULL;
    }
    else if (x>t->data)
    {
        t->right=insert(t->right,x);

    }
    else if (x<t->data)
    {
        t->left=insert(t->left,x);
    }
    return t;
}

void inorder(struct tree *t)
{
    if (t->left!=NULL)
        inorder(t->left);
    printf("%d ",t->data);
    if (t->right!=NULL)
        inorder(t->right);
}

void preorder(struct tree *t){
printf("%d ",t->data);
if(t->left!=NULL)
    preorder(t->left);
if(t->right!=NULL)
    preorder(t->right);

}

void postorder(struct tree *t){
```

```
if(t->left!=NULL)
    postorder(t->left);
if(t->right!=NULL)
    postorder(t->right);
printf("%d ",t->data);
}



void printname(struct tree *t,int x)
{
    for (int i = 0;i<x;i++)
    {
        printf("\t");
    }
    printf("%d\n",t->data);

}

void levelorder(struct tree *t,int x)
{
    printname(t,x);
    if (t->left!=NULL)
        levelorder(t->left,x+1);
    if (t->right!=NULL)
        levelorder(t->right,x+1);
}


struct tree* findmin(struct tree *t)
{
    if (t->left==NULL)
        return t;
    else
        findmin(t->left);
}

int search(struct tree *t,int key)
{
    if (t!=NULL)
    {
        if (key==t->data)
            return 1;
        else if(key<t->data)
            search(t->left,key);
        else if (key>t->data)
            search(t->right,key);

    }
    else
        return 0;
```

```
}


struct tree *delete(struct tree *t,int x)
{
    struct tree *tmp;
    if (x<t->data)
        t->left=delete(t->left,x);
    else if (x>t->data)
        t->right=delete(t->right,x);
    else if (t->left &&t->right)
    {
        tmp=findmin(t->right);
        t->data=tmp->data;
        t->right=delete(t->right,tmp->data);
    }
    else
    {
        tmp=t;
        if (t->right==NULL)
            t=t->left;
        else if (t->left==NULL)
            t=t->right;
        free(tmp);
    }
    return t;
}
```

Main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "bstadt.h"

void main()
{
    struct tree *t;
    t=NULL;
    printf("enter the number of nodes :");
    int n;
    scanf("%d",&n);
    for (int i = 0;i<n;i++)
    {
        printf("enter the value :");
        int a;
        scanf("%d",&a);
        t=insert(t,a);
    }
    printf("--INORDER--\n");
    inorder(t);
```

```
    printf("\n--PREORDER---\n");
    preorder(t);
    printf("\n--POSTORDER---\n");
    postorder(t);
    printf("\n--LEVEL ORDER--\n");
    levelorder(t,0);
    printf("\n--FIND MINIMUM--\n");
    struct tree *res = findmin(t);
    printf("MINIMUM : %d",res->data);
    printf("\n--SEARCH--\n");
    printf("enter the key :");
    int key;scanf("%d",&key);
    int r=search(t,key);
    if (r==1)
        printf("ELEMENT PRESENT!\n");
    else
        printf("ELEMENT IS NOT PRESENT!\n");
    printf("\n--DELETION--\n");
    printf("enter the element to be deleted:");
    int del;scanf("%d",&del);
    delete(t,del);
    printf("\n--AFTER DELETION--\n");
    printf("--INORDER--\n");
    inorder(t);
    printf("\n--PREORDER---\n");
    preorder(t);
    printf("\n--POSTORDER---\n");
    postorder(t);
    printf("\n--LEVEL ORDER--\n");
    levelorder(t,0);
}
```

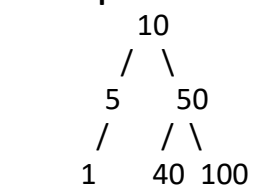
NAME : S.KARTHIKEYAN 3122 22 5001 056
CSE B

OUTPUT:

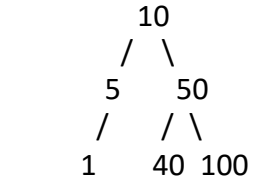
```
enter the number of nodes :4
enter the value :10
enter the value :5
enter the value :17
enter the value :67
--INORDER--
5 10 17 67
--PREORDER---
10 5 17 67
--POSTORDER---
5 67 17 10
--LEVEL ORDER--
10
        5
        17
                67

--FIND MINIMUM--
MINIMUM : 5
--SEARCH--
enter the key :17
ELEMENT PRESENT!

--DELETION--
enter the element to be deleted:17

--AFTER DELETION--
--INORDER--
5 10 67
--PREORDER---
10 5 67
--POSTORDER---
5 67 10
--LEVEL ORDER--
10
        5
        67
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

Test case for the Application
(a)
Input: Tree 1



Input: Tree2



Tree1 and Tree2 are identical with a set of elements
(b)
Tree1 not
complete(c)
Tree1 Range: [**5**, 45]
Output: 3
Nodes are 5, 10, 40

Tree2 Range: [**1**, 45]
Output: 4
Nodes are 1,5, 10, 40

Given two binary search trees `root1` and `root2`, return *a list containing all the integers from both trees sorted in* ***ascending*** *order*.

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

2. Write an application to do the following
   a. Check whether the two BST contains the same set of elements

ALGORITHM:
FUNCTION NAME:check
INPUT:
i)struct tree *
ii)struct tree *
OUTPUT:int

1. if (t->left!=NULL && t1->left!=NULL)
   check(t->left,t1->left);
2. if (t->data != t1->data)
   return 1;
3. if (t->right!=NULL && t1->right!=NULL)
   check(t->right,t1->right);

OUTPUT:

```
TREE 1
enter the number of nodes :4
enter the value :10
enter the value :4
enter the value :5
enter the value :100
--INORDER--
4 5 10 100
--LEVEL ORDER--
10
        4
                5
        100


TREE 2
enter the number of nodes :4
enter the value :100
enter the value :4
enter the value :5
enter the value :10
--INORDER--
4 5 10 100
--LEVEL ORDER--
100
        4
                5
                        10

--CHECKING---
NODE OF TREE 1 : 4
NODE OF TREE 2 : 4
TREES ARENT IDENTICAL
```

```
TREE 1
enter the number of nodes :4
enter the value :1
enter the value :2
enter the value :3
enter the value :4
--INORDER--
1 2 3 4
--LEVEL ORDER--
1
        2
                3
                        4


TREE 2
enter the number of nodes :4
enter the value :1
enter the value :2
enter the value :3
enter the value :4
--INORDER--
1 2 3 4
--LEVEL ORDER--
1
        2
                3
                        4

--CHECKING---
NODE OF TREE 1 : 4
NODE OF TREE 2 : 4
TREES ARE IDENTICAL
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

b. Count the number of nodes in tree within the given range

FUNCTION NAME: num
INPUT:
i)struct tree *
ii)int
iii)int
OUTPUT:int

```
1.  static int n=0;
2.   if (t==NULL)
      return n;
3.   else
    {
        if (t->data>=l && t->data<=h)
        {
            n+=1;
            printf("%dst NODE :",n);
            printf(" %d\n",t->data);

            num(t->left,l,h);
            num(t->right,l,h);

        }
        if (t->data<l)
        {
            num(t->right,l,h);
        }
        if (t->data>h)
        {
            num(t->left,l,h);
        }
  }
```

OUTPUT:

```
TREE 1
enter the number of nodes :5
enter the value :10
enter the value :5
enter the value :19
enter the value :78
enter the value :67
--INORDER--
5 10 19 67 78
--LEVEL ORDER--
10
        5
        19
                78
                        67

--RANGE--
enter the lower value of range :10
enter the higher value of range :67
--NUMBER OF NODES WITHIN THE RANGE
1st NODE : 10
2st NODE : 19
3st NODE : 67
```

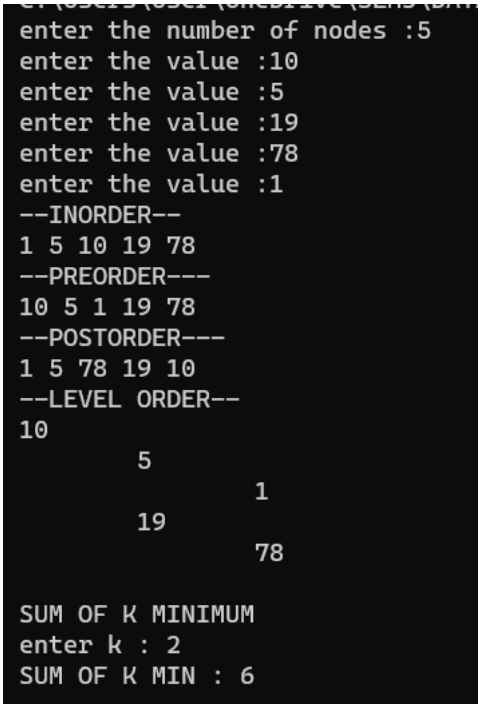
NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

c. Find sum of k smallest elements in the given BST

FUNCTION NAME:sum
INPUT:
i)struct tree *
ii)int
OUTPUT:int

```
1. static int s=0;static int n=0;
2. if (t->left!=NULL)
       sum(t->left,k);
3. if (n!=k)
       {
       s+=t->data;
       n++;}
4. if (n==k)
       return s;
5. if (t->right!=NULL)
       sum(t->right,k);
```

OUTPUT:

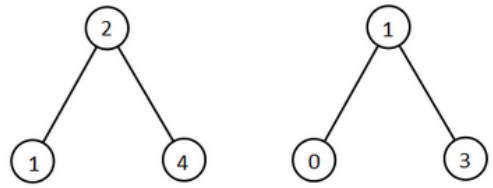


NAME : S.KARTHIKEYAN **3122 22 5001 056**

CSE B

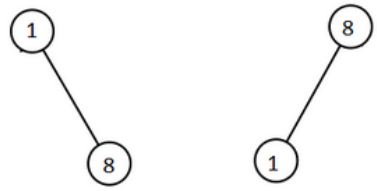
QUESTION 3:

Example 1:




```
Input: root1 = [2,1,4], root2 = [1,0,3]
Output: [0,1,1,2,3,4]
```

Example 2:




```
Input: root1 = [1,null,8], root2 = [8,1]
Output: [1,1,8,8]
```

ALGORITHM:

FUNCTION NAME: sort

INPUT:

i)int []

ii)int[]

OUTPUT: int *

```
1. int i=0,j=0,k=0;
2. while(i<n1 && j<n2)
   {
        if (root1[i]<root2[j])
        {
             res[k]=root1[i];
              i++;
             k++;
        }
         else
        {
```

```
            res[k]=root2[j];
            j++;
            k++;
            }
    }
```

3. while (i<n1)

```
    {
            res[k]=root1[i];
            k+=1;i+=1;
    }
```

4. while (j<n2)

```
    {
            res[k]=root2[j];
            k++;j++;
    }
```

5. return res;

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

OUTPUT:

```
TREE 1
enter the number of nodes :4
enter the value :10
enter the value :11
enter the value :7
enter the value :4
--INORDER--
4 7 10 11
--LEVEL ORDER--
10
        7
                4
        11


TREE 2
enter the number of nodes :3
enter the value :9
enter the value :16
enter the value :86
--INORDER--
9 16 86
--LEVEL ORDER--
9
        16
                86

--SORTED--
4 7 9 10 11 16 86
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

QUESTION 4: Write a function to check whether the given tree is BST

FUNCTION NAME: check

INPUT:

i)struct tree *

OUTPUT: int

```
1. if (t==NULL)
        return 1;
2. if (t->right==NULL && t->left==NULL)
        return 1;
3. int lmax,rmin;
4. if (t->left!=NULL)
        lmax=maxinleft(t->left);  //returns max value in left subtree of  a node
5. else
        lmax=0;
6. if (t->right!=NULL)
        rmin=mininright(t->right);   //returns the minimum value in the right subtree
of a node
7. else
        rmin=t->data;

8. if (lmax>t->data || t->data > rmin)
        {
            return 0;
         }
9. int a,b;
10. a=checkbst(t->left);
11. b=checkbst(t->right);
12. if (a==1 && b==1)
         return 1;
13. else
        return 0;
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DA
enter the number of nodes :4
enter the value :5
enter the value :3
enter the value :89
enter the value :56
--INORDER--
3 5 56 89
--PREORDER---
5 3 89 56
--POSTORDER---
3 56 89 5
--LEVEL ORDER--
5
        3
        89
                56

--CHECK BST--
ANSWER : 1
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE B